Applications graphiques

Rappels de traitements d'images & OpenCV

Hugo Rositi

Saint-Dié-des-Vosges, le 21 septembre 2026

hugo.rositi@univ-lorraine.fr

IUT Saint-Dié-des-Vosges

UNIVERSITÉ DE LORRAINE

Organisation

- CM1 : Rappels et présentation OpenCV (21/09)
  - TD1 : Bases du TI avec OpenCV (23/09)
  - TP1 : OpenCV.JS et autres (23/09)
- TD2/CM2 : Segmentation d'images (15/10)
  - TP2 : Seuillage, hysterésis, k-means (15/10)
  - TD3 : Approche région (watershed) (20/10)
- TD3 : Flot optique (20/10)
- TP3 : Mini-projet en binôme (20/10)
- TD4 : 1h évaluation + 1h projet (22/10)
- TP4 : Soutenance du projet (22/10)

Motivation

Pourquoi faire du TI classique à l'heure de l'intelligence artificielle ?

Pourquoi faire du traitement d'images classique à l'heure de l'intelligence artificielle ?

C'est une excellente question, Hugo. Même à l'ère de l'intelligence artificielle (IA), le traitement d'images classique reste pertinent et complémentaire pour plusieurs raisons :

---

1. Efficacité et rapidité

- **Faible latence** : Les méthodes classiques (filtrage, seuillage, morphologie mathématique, etc.) sont souvent plus rapides et moins gourmandes en ressources que les modèles d'IA, surtout pour des tâches simples ou en temps réel (ex : traitement vidéo embarqué, systèmes de surveillance).
- **Pas d'apprentissage nécessaire** : Pas besoin de données d'entraînement ou de calculs lourds pour appliquer un filtre de flou ou détecter des contours.

Notion d'explicabilité

Connaissance exacte de l'impact du traitement sur les données en entrée.
Exemple d'un débruitage avec IA ou par une approche filtrage

Motivation

Pourquoi faire du TI classique à l'heure de l'intelligence artificielle ?

3. Complémentarité avec l'IA

- **Prétraitement** : Les méthodes classiques sont souvent utilisées en amont des modèles d'IA pour améliorer la qualité des données (ex : correction de la luminosité, suppression du bruit, recadrage).
- **Post-traitement** : Après une prédiction par IA, des techniques classiques peuvent affiner les résultats (ex : morphologie mathématique pour nettoyer une segmentation).
- **Hybridation** : Certains systèmes combinent les deux approches pour tirer parti de leurs forces respectives (ex : détection de visages par IA + suivi par méthodes classiques).

4. Robustesse dans des contextes spécifiques

- **Environnements contraints** : Dans des systèmes embarqués (ex : capteurs industriels, drones), où la puissance de calcul est limitée, les méthodes classiques sont souvent préférées.
- **Données limitées** : Si on dispose de peu de données annotées, les méthodes classiques peuvent être plus fiables qu'un modèle d'IA sous-entraîné.

Motivation

Pourquoi faire du TI classique à l'heure de l'intelligence artificielle ?

6. Cas d'usage où l'IA n'est pas nécessaire

- **Tâches simples** : Pour des opérations comme le redimensionnement, la rotation, la conversion de couleurs, ou l'application de masques, les méthodes classiques suffisent largement.
- **Normes industrielles** : Certaines industries (ex : imagerie médicale, contrôle qualité) utilisent des protocoles standardisés basés sur des méthodes classiques, validés depuis des décennies.

Exemples concrets

Tâche	Méthode Classique	IA
Détection de contours	Filtrage de Sobel/Canny	Réseau de neurones (ex : U-Net)

Motivation

Pourquoi faire du TI classique à l'heure de l'intelligence artificielle ?

Notion d'explicabilité

Connaissance exacte de l'impact du traitement sur les données en entrée. Exemple d'un débruitage avec IA ou par une approche filtrage.

exemple du militaire ou d'autres domaines sensibles (médical)

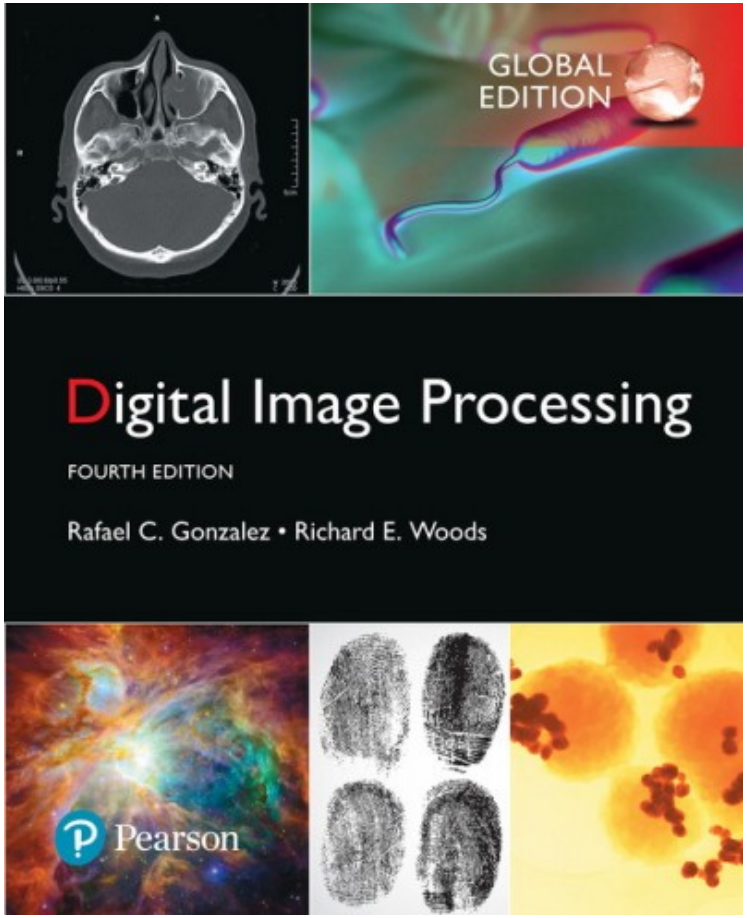
Pourquoi faire du TI classique à l'heure du machine learning ?

Le traitement d'images classique n'est pas obsolète : il reste indispensable pour des tâches spécifiques, en complément de l'IA, ou dans des contextes où ses avantages (rapidité, simplicité, transparence) sont décisifs. L'idéal est souvent de combiner les deux approches pour obtenir le meilleur des deux mondes.

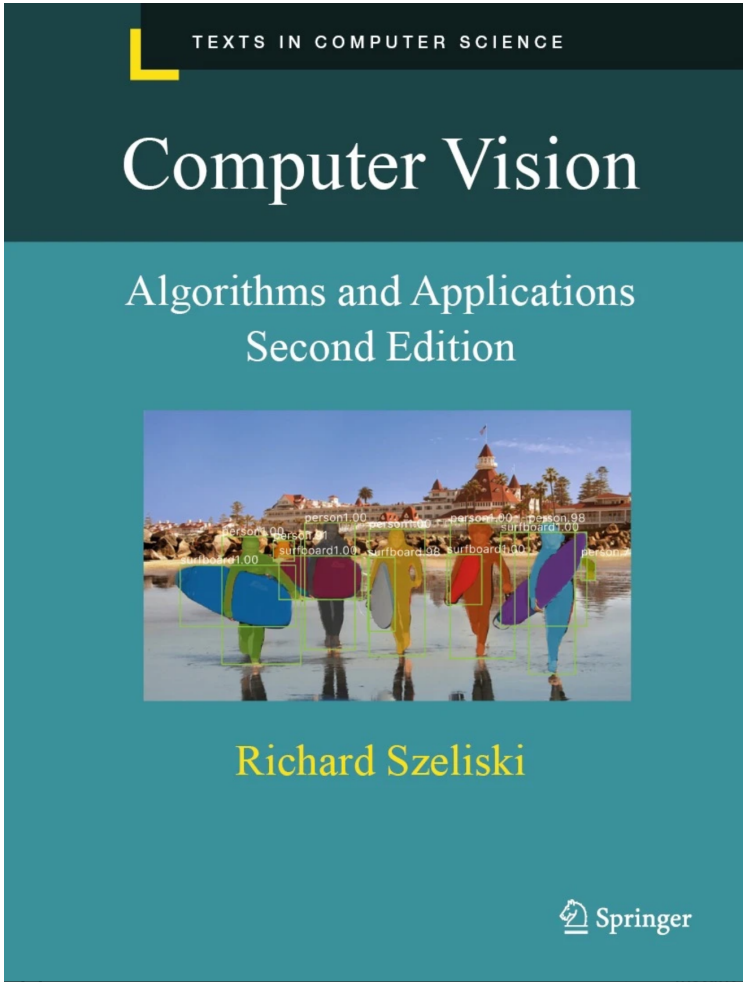
Motivation (2)

- Choix de ne pas aborder la partie IA du traitement d'images moderne.
- Champ disciplinaire vu dans d'autres modules.
- Si vous poursuivez dans le domaine de l'image, cela vous sera introduit.

Deux références indispensables



Des matériaux supplémentaires



Téléchargement légal

Domaines applicatifs



Image satellite



Image satellite agricole

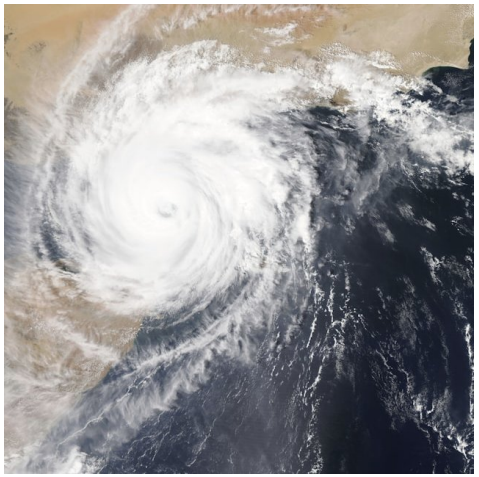


Image satellite météo

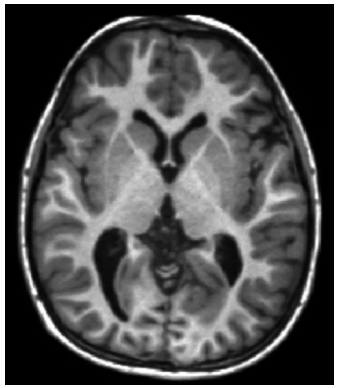
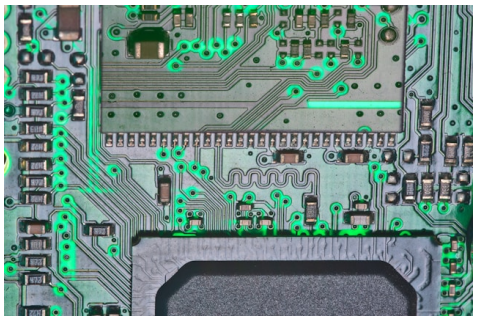
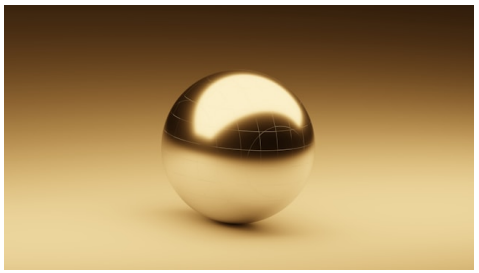


Image médicale

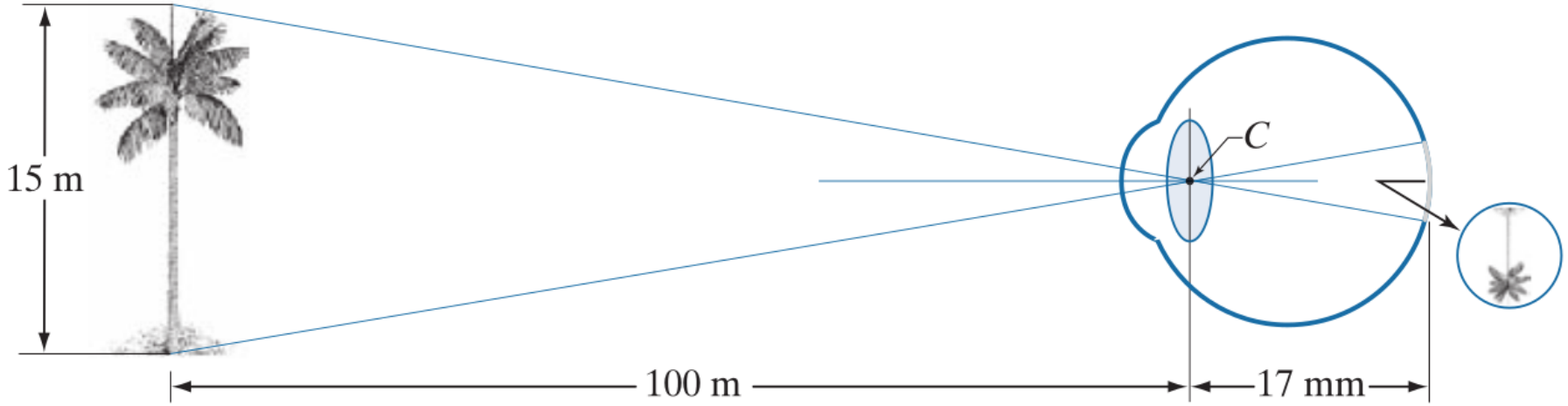


Contrôle industriel



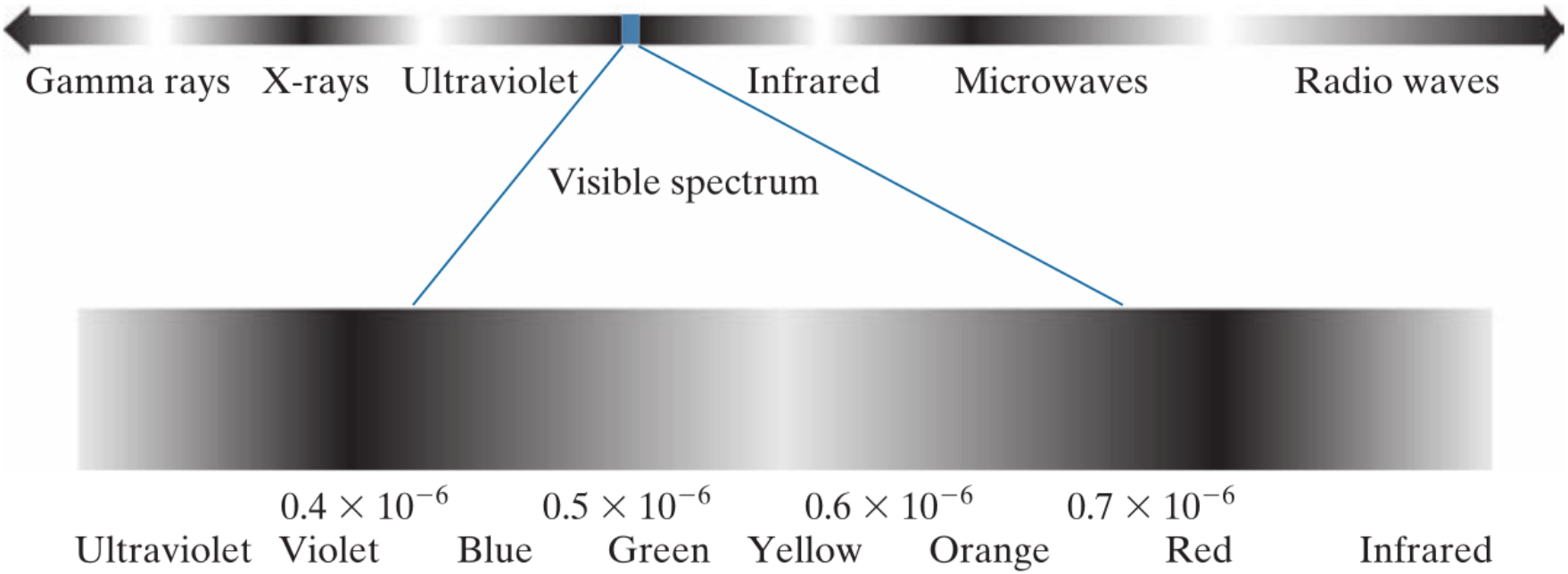
Photographie

Fondamentaux de la vision



La vision par ordinateur se calque en grande partie sur la perception visuelle humaine.

Fondamentaux de la vision

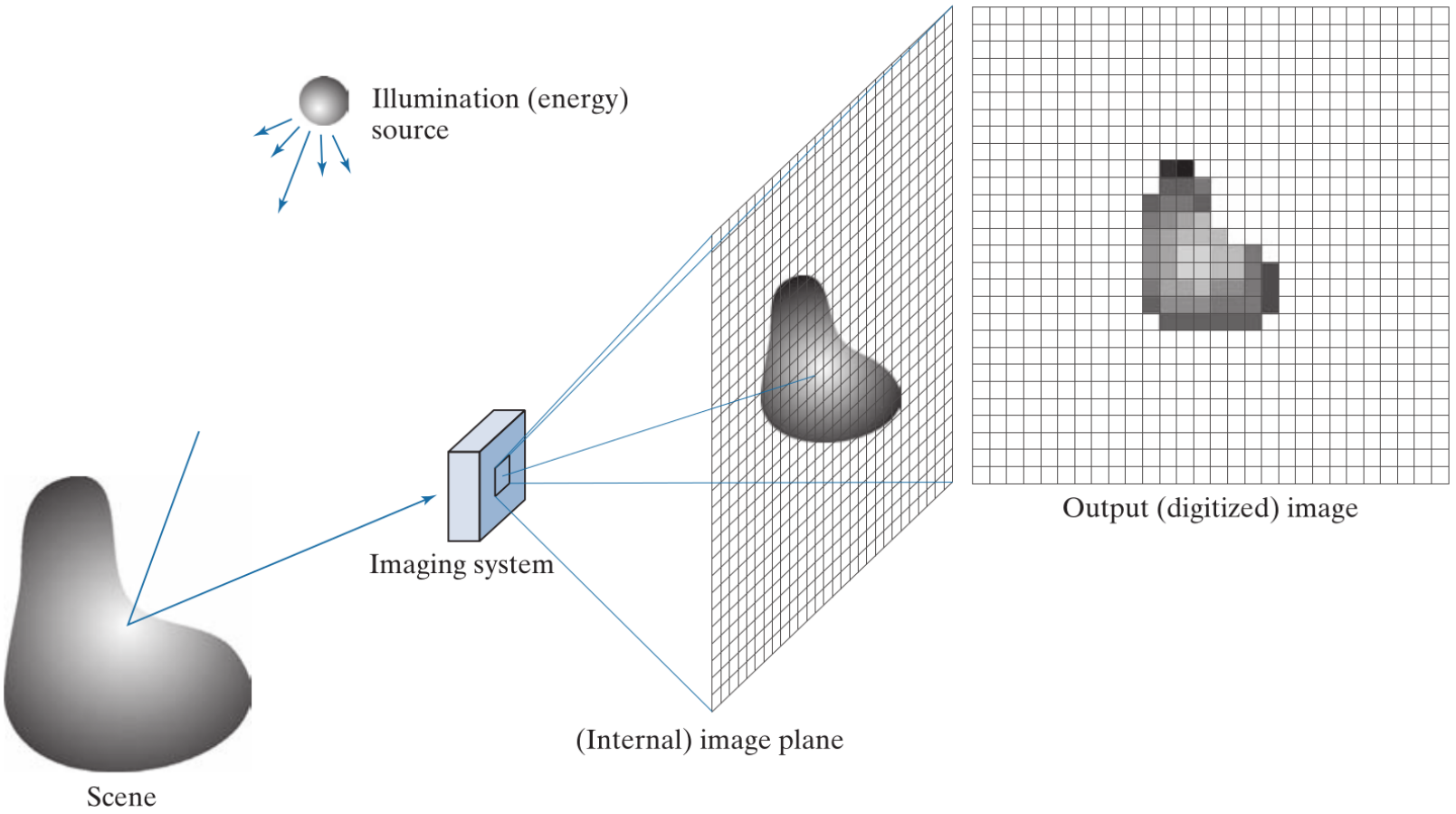


Les interactions avec toute la gamme du spectre électromagnétique permettent de mettre en avant différents objets ou particularités des objets observés.

Formation d'une image numérique

Formation d'une image numérique

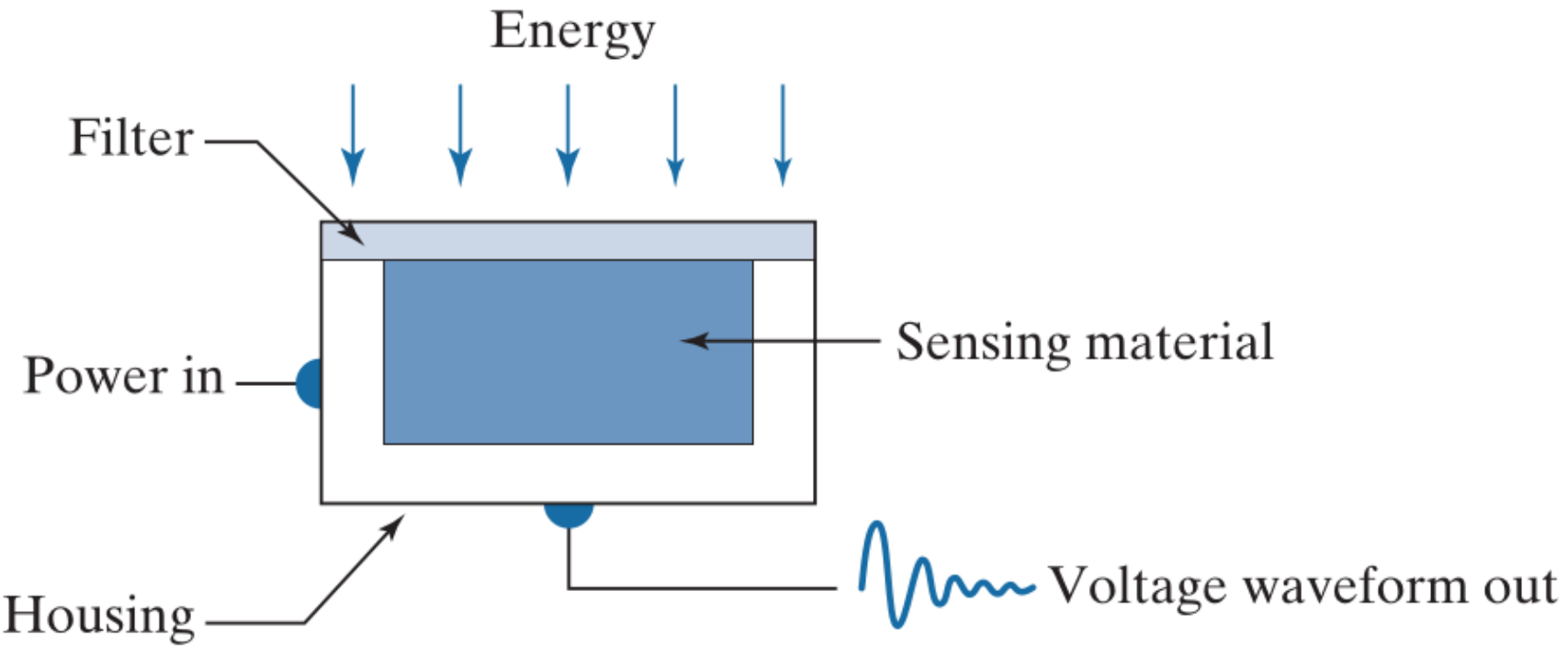
Une image est construite à partir de l'interaction entre plusieurs éléments clés : la lumière, les objets, l'optique des caméras et les capteurs.



Formation d'une image numérique

Formation d'une image numérique

Une image est construite à partir de l'interaction entre plusieurs éléments clés : la lumière, les objets, l'optique des caméras et les capteurs.

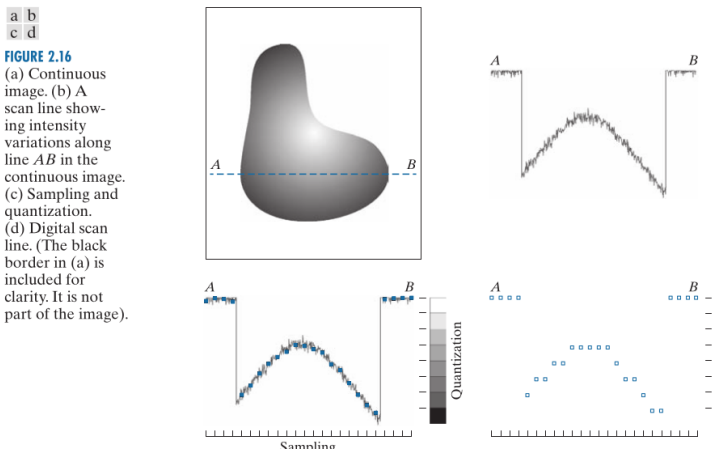


Formation d'une image numérique

Une image numérique

Une image numérique est le résultat de la conversion de photons en plusieurs signaux électriques. Cette étape clé d'**acquisition** est décomposée en deux sous-étapes :

- l'**échantillonage** : transformer un signal continu en signal discret du point de vue spatial.
- la **quantification** : transformer un signal continu en signal discret du point de vue de son amplitude (intensité).



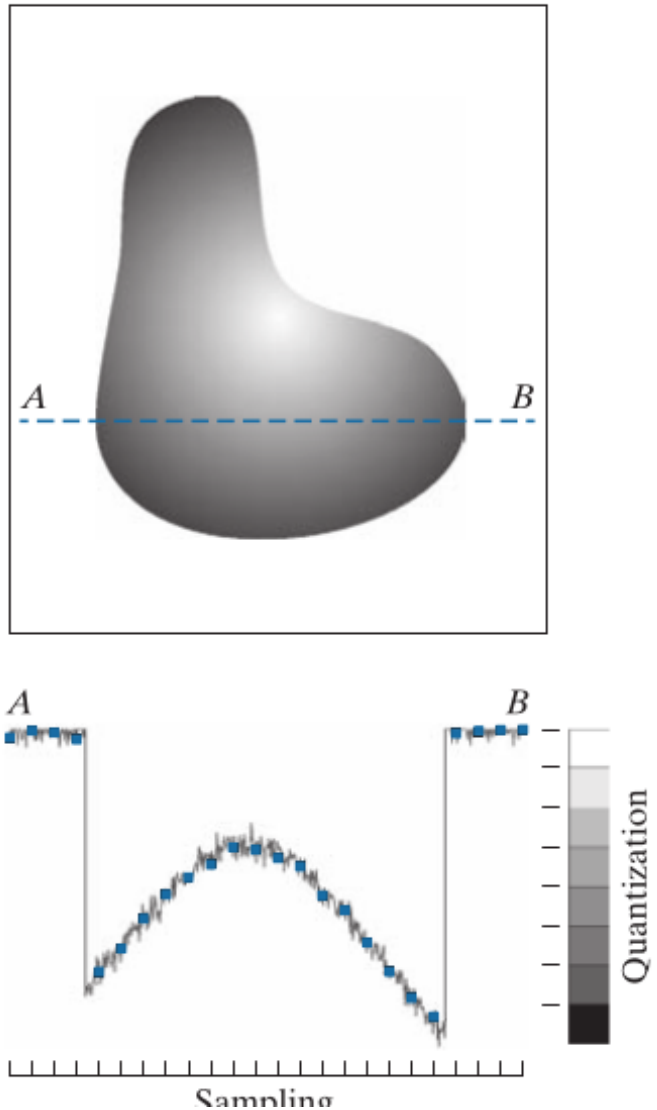
a b
c d

FIGURE 2.16
(a) Continuous image. (b) A scan line showing intensity variations along line AB in the continuous image. (c) Sampling and quantization. (d) Digital scan line. (The black border in (a) is included for clarity. It is not part of the image).

Formation d'une image numérique

a b
c d

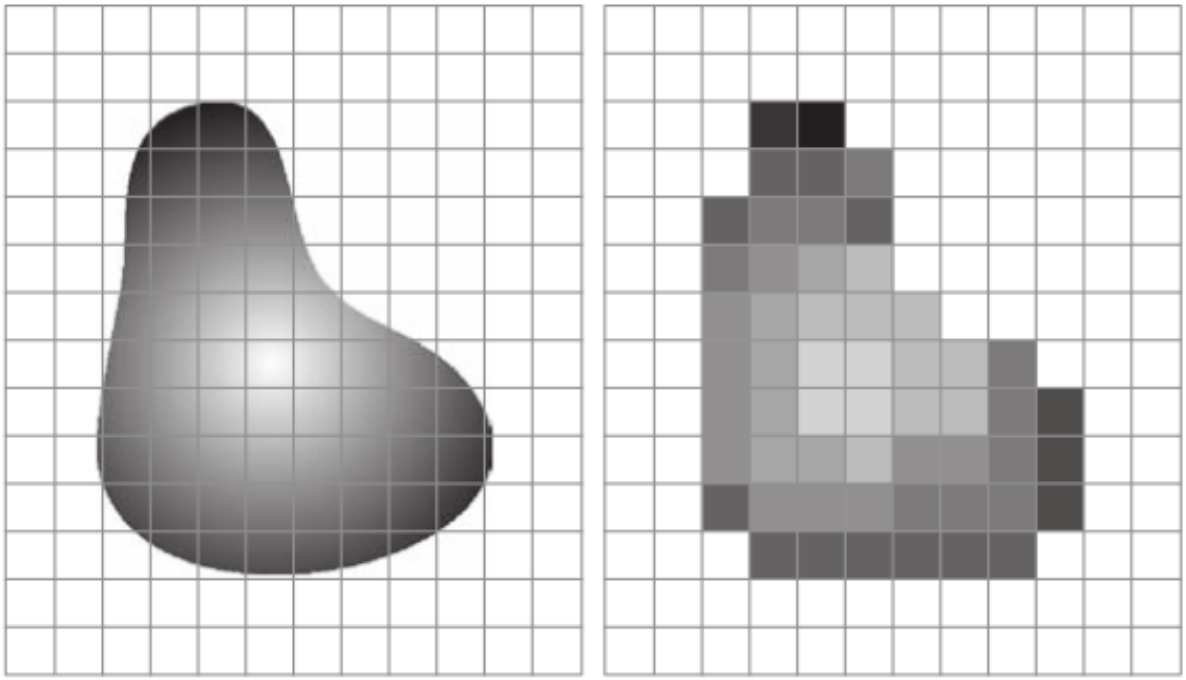
FIGURE 2.16
(a) Continuous image. (b) A scan line showing intensity variations along line AB in the continuous image. (c) Sampling and quantization. (d) Digital scan line. (The black border in (a) is included for clarity. It is not part of the image).



Formation d'une image numérique

a b

FIGURE 2.17
(a) Continuous image projected onto a sensor array. (b) Result of image sampling and quantization.



Biais de la vision humaine

Biais de la vision humaine

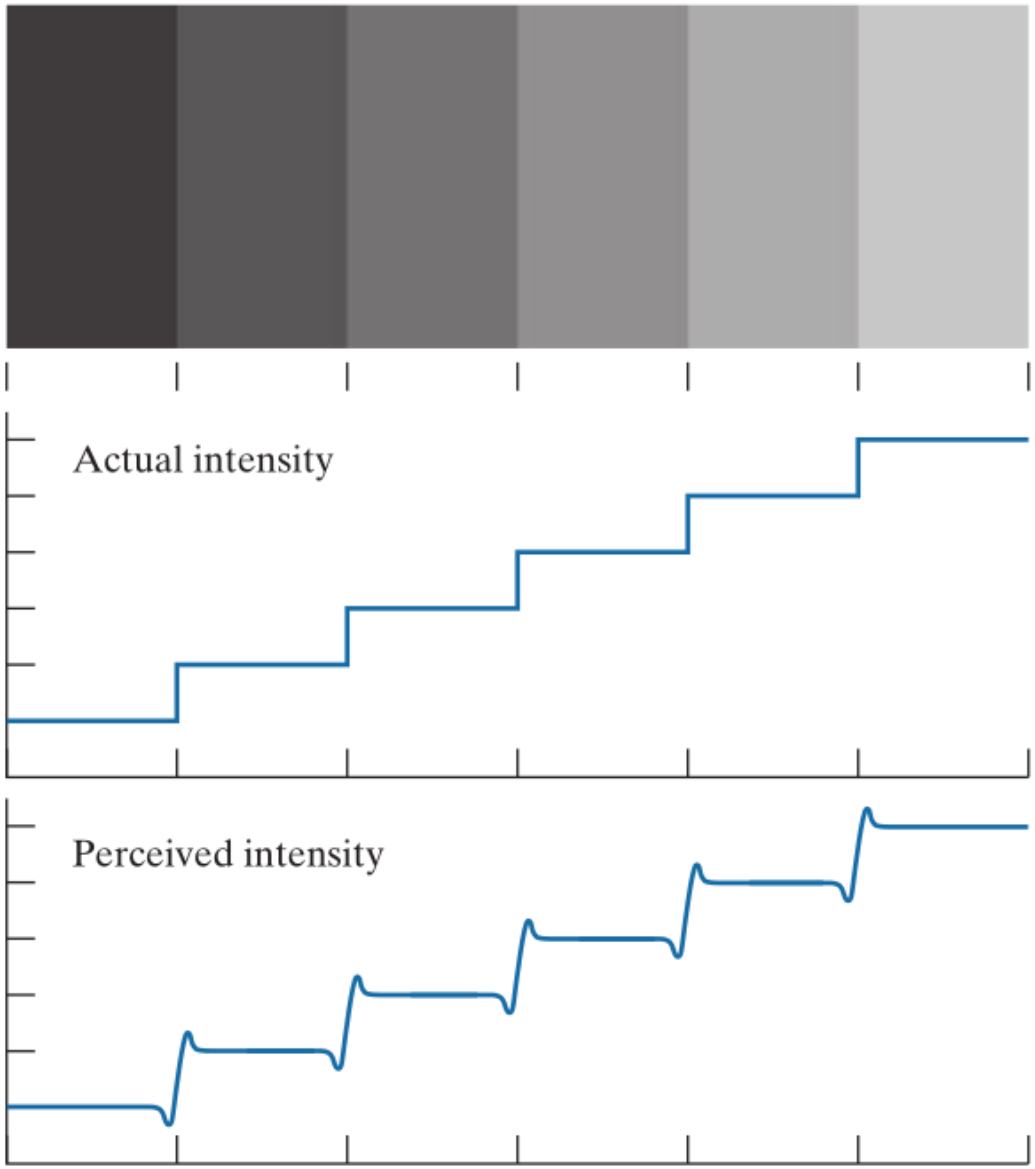
- Large spectre de discrimination des intensités de l'ordre de $10^1 0$.
- En contrepartie : la fenêtre de discrimination simultanée des intensités est quant à elle relativement étroite.
- La sensibilité globale de l'œil est ainsi modifiée.



a b c

FIGURE 2.8 Examples of simultaneous contrast. All the inner squares have the same intensity,

Biais de la vision humaine (2)



Biais de la vision par ordinateur

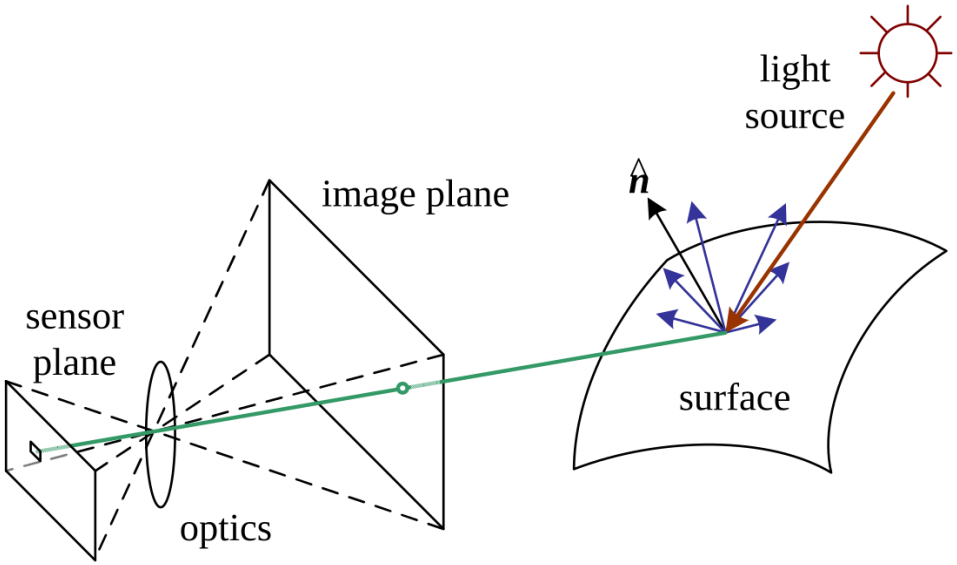
Biais de la vision par ordinateur

- Dans un système ou on voudrait tenir compte des mesures réelles des objets.
- Les caractéristiques de la géométrie des caméras et les propriétés des capteurs sont **fondamentales**.
- Il faut également tenir compte de différents biais :
  - Aberration chromatique
  - Aberration sphérique
  - Projection des objets (3D $\rightarrow$ 2D)
  - Déformations spatiales (lentilles optiques)
  - Vignettage (chute du contraste aux bords)
  - Modèle d'illumination (radiance, réfléctance, diffusion, *etc.*)

Notions plutôt abordées pour la synthèse d'images.

Lumière et formation de l'image

→ Les images se forment grâce à l'interaction entre les objets de la scène et une source de lumière. La lumière incidente est réfléchie, transmise à un capteur et forme ainsi l'image.





Interactions de la lumière

- Absorption
- Diffusion
- Réflexion

↪ La lumière est une radiation composée de différentes longueurs d'ondes, leurs interactions seront variables.

Représentation des images : contraste et couleurs

L'oeil humain

L'oeil humain perçoit les contrastes et les différentes couleurs grâce à deux éléments constituants de la rétine. Les **cônes** et les **batonnets** sont présents en différentes proportions à sa surface et permettent respectivement une vision **photopique** et **scotopique**.

Perception des couleurs

- Les cônes sont divisés en trois catégories chacune sensible à une gamme différente du spectre EM.
  - Le Rouge : $\lambda = 700\ nm$
  - Le Vert : $\lambda = 546.1\ nm$
  - Le Bleu : $\lambda = 435.8\ nm$

Représentation des couleurs

D'autres espaces de couleurs

- RGBa (synthèse additive)
- YCbCr
- Hue, Saturation, Value
- CMYK (synthèse soustractive)

Il faut distinguer la chrominance de la luminance. L'oeil humain est plus sensible aux contrastes qu'à la couleur. On peut passer d'un espace à un autre.

Représentation des couleurs (2)



(a) RGB



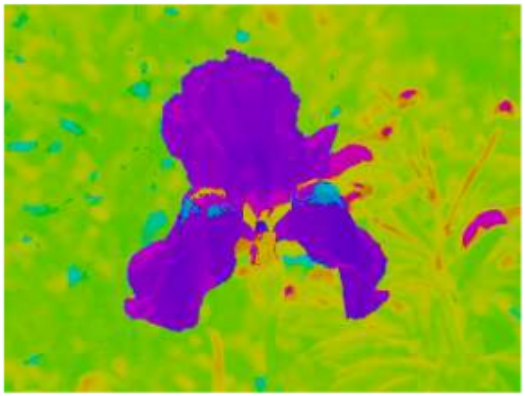
(b) R



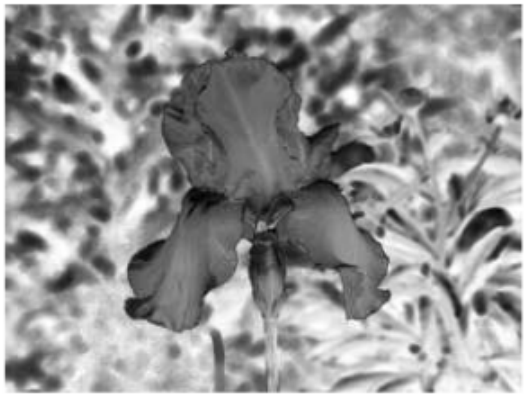
(c) G



(d) B



(l) H



(m) S



(n) V

Format de fichiers et compression

Définitions

- *Format de fichiers* : Standard pour organiser et stocker les données images.
- *Conteneur* : Similaire aux formats mais permet de gérer différents types de données (*e.g.* plusieurs images ; tiff, gif, DICOM).
- *Standards de compression* : Méthodes pour compresser et décompresser des données.

Format de fichiers et compression

Compression

Réduire ou supprimer une ou plusieurs redondances pour compresser.
Redondance spatiale et temporelle.

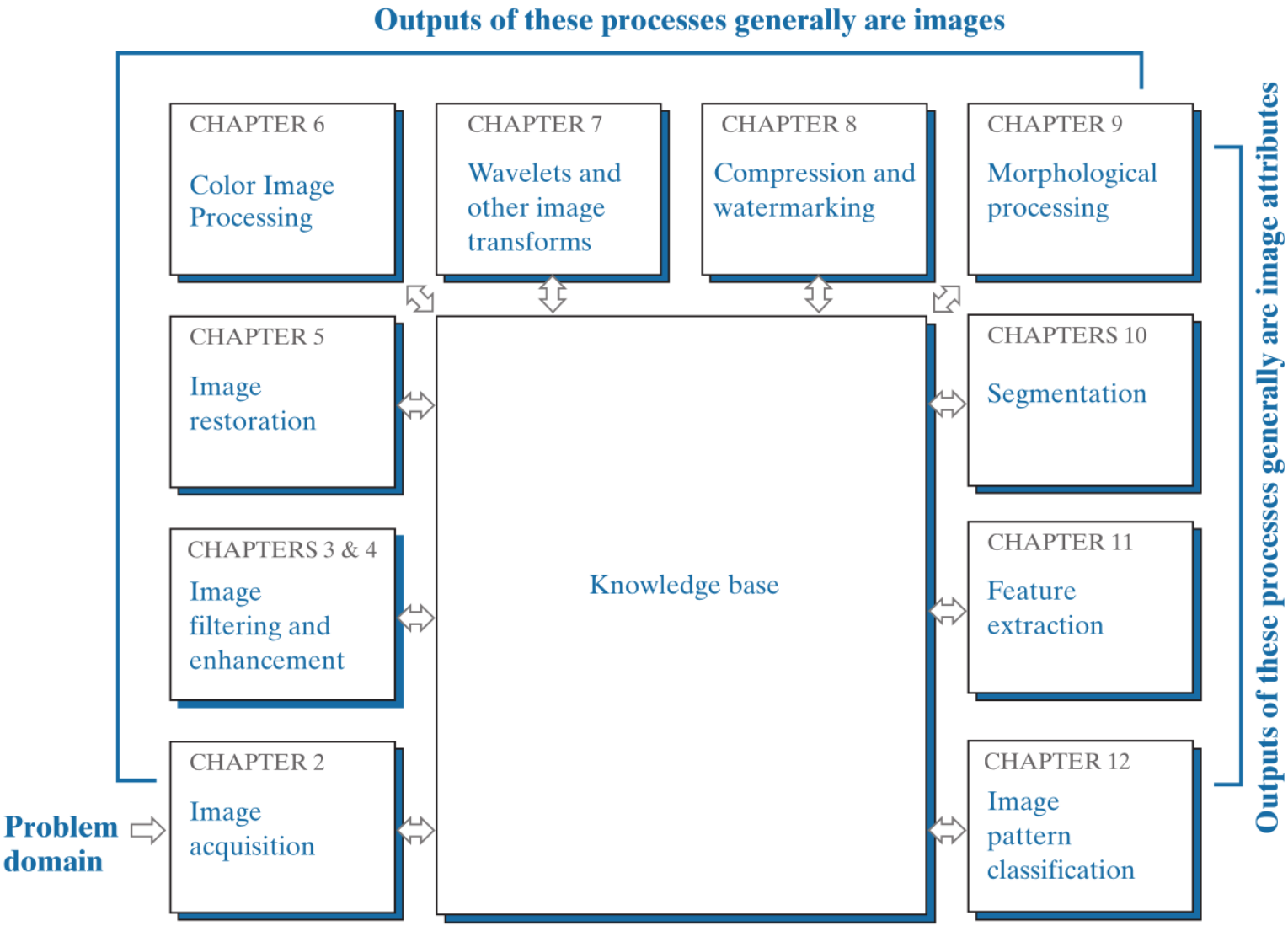
- Huffman coding : Les données les plus fréquentes sont codées avec des codes courts.
- Golomb coding : Efficace pour les données où les petites valeurs sont plus nombreuses (Exponentielle décroissante)
- Arithmetic coding : Remplace une séquence de symboles par un nombre réel.
- LZW coding : assigne un mot de longueur fixe à des séquences de symboles source variables en longueur.

Différents formats de fichiers

Nom	Type de données	Type de compression
BMP	Image	Pas de compression
JPEG	Image	Avec pertes, DCT, Huffman, Run-length
PNG	Image	Sans pertes, probabiliste
TIFF	Image	Différentes standards supportés
DV	Video	DCT approach
H.26X	Video	Variable block size, Prediction differences, arithmetic coding
MPEG	Video	Similaire H.26X

Traitement d'images (*Image processing*)

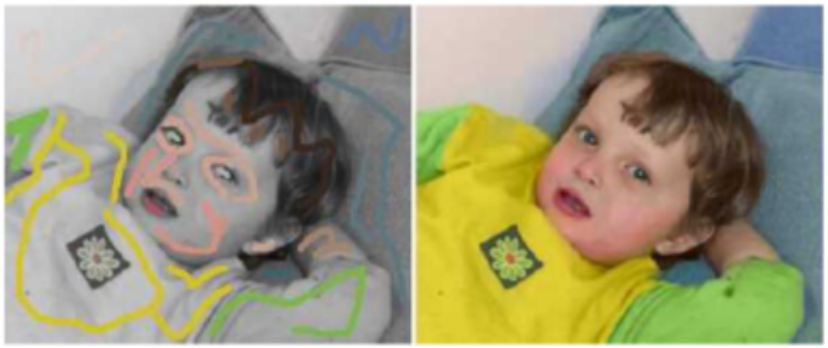
Que peut-on faire comme traitements sur une image ?



Vision par ordinateur (*Computer vision*)



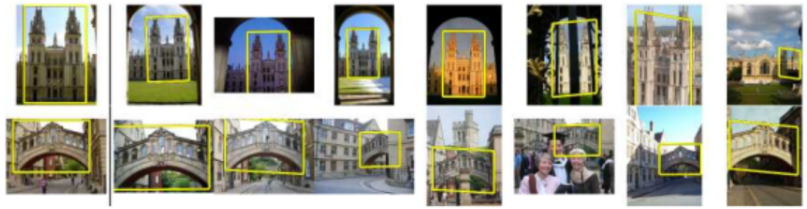
Segmentation.



Colorisation d'images.



Appariement d'images.



Détection de structures.

Vision par ordinateur (*Computer vision*)

À partir de ces différents traitements, on va pouvoir dériver des applications plus avancées de vision par ordinateur dans de très nombreux domaines industriels (*e.g.* agriculture, divertissement, défense, médical, *etc.*).

D'autres applications

- Segmentation interactive
- Reconnaissance de formes. (*e.g.* panneau de circulation)
- Estimation de posture. (*e.g.* Reconnaissance gestuelle)
- Rendu de scène 3D à partir d'images 2D. (*e.g.* image satellitaire)
- Fusion d'images. (*e.g.* création de panoramique)
- Amélioration d'images. (*e.g.* inpainting)
- Estimation de mouvements.
- Classification d'images.
- *etc.*

Transformations d'images

Opérateurs locaux

- Se base sur la seule valeur d'intensité du pixel.

→ Amélioration du contraste de l'image (égalisation d'histogramme).

Filtrage linéaire

- Tient compte de l'environnement local du pixel (voisinage).

→ Réduction du bruit dans l'image (*noise reduction*).

Traitements d'images binaires

- Traitement de l'image sur deux niveaux d'intensités : morphologie mathématique.

→ Amélioration de masque binaire après seuillage.

Transformations de domaine

- Changement de représentation de l'image : domaine fréquentiel.

→ Réduction du flou dans l'image (*sharpening*).

- Transformations géométriques

Opérateurs locaux

Transformations basées sur l'intensité des pixels

- Transformations des intensités (multiplication et addition de constantes).
- Transformations des couleurs (map une couleur vers une autre).
- Composition d'images
- Modifications des intensités en lien avec l'histogramme.

Compositing and matting

Composition

L'équation suivante permet de combiner deux images ou deux-sous régions et de les superposer l'une considérer comme avant-plan et l'autre comme arrière-plan.

$Comp(x) = (1 - \alpha)f_0(x) + \alpha f_1(x)$, avec $\alpha \in [0, 1]$

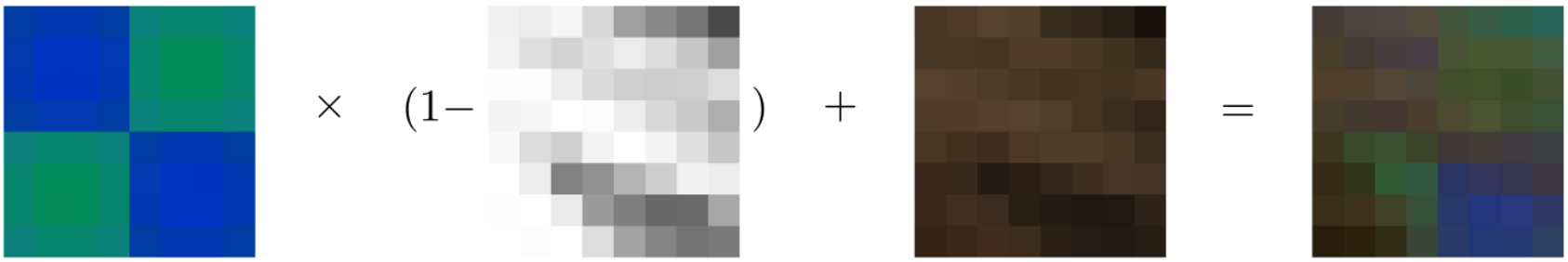


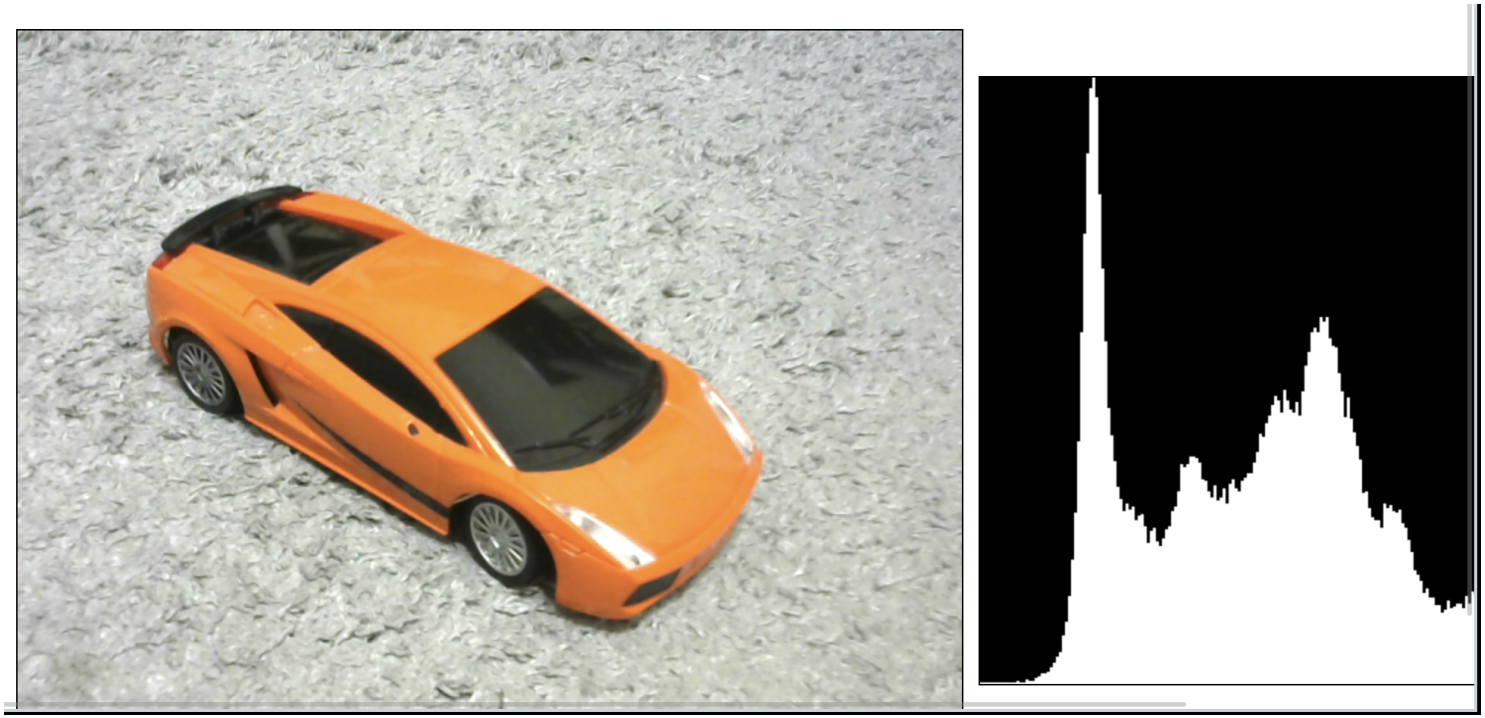
Figure 3.5 *Compositing equation $C = (1 - \alpha)B + \alpha F$. The images are taken from a close-up of the region of the hair in the upper right part of the lion in Figure 3.4.*

↪Implémentation OpenCV.js

Histogramme

Histogramme

L'histogramme permet de représenter sous la forme d'un graphique la répartition des intensités d'une image. Pour les images couleurs, on traitera un histogramme pour chaque canal de couleurs.

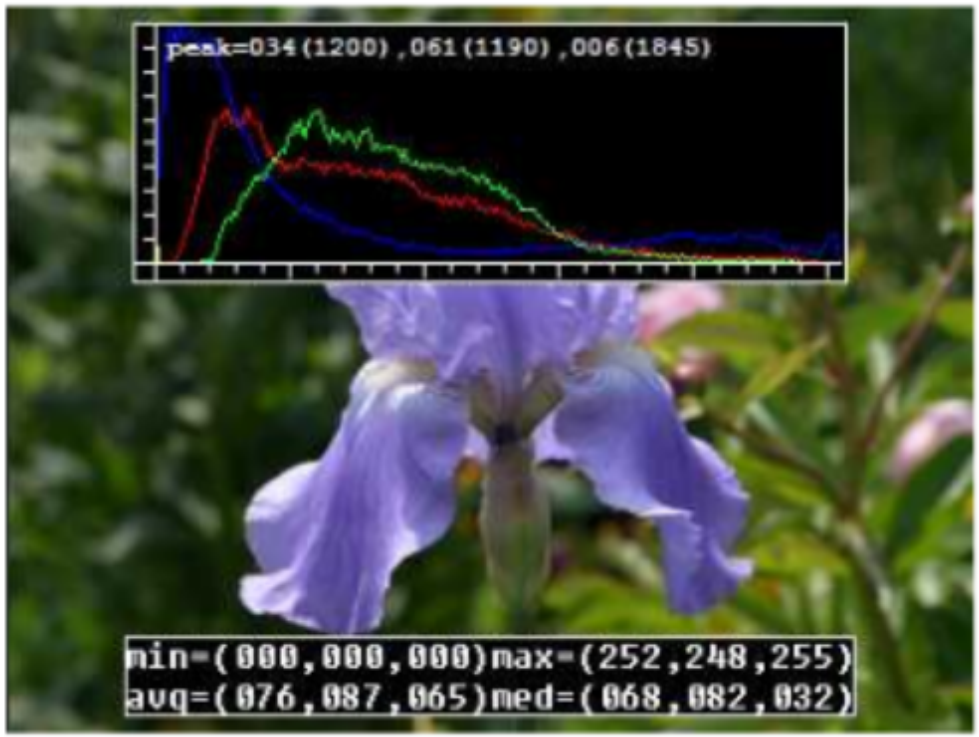


↪ Implémentation OpenCV.JS

Histogramme

Histogramme

L'histogramme permet de représenter sous la forme d'un graphique la répartition des intensités d'une image. Pour les images couleurs, on traitera un histogramme pour chaque canal de couleurs.



↪ Implémentation OpenCV.JS

Égalisation d'histogramme

Égalisation d'histogramme

Pour améliorer le contraste d'une image, le travail sur l'histogramme est une solution. L'effet recherché est ici d'associer la valeur la plus sombre de l'image à la valeur noire (0) et la plus claire au blanc pur (255, sur 8 bits).

L'opération

$$c(I) = \frac{1}{N} \sum_{i=0}^{I} H(i) = c(I-1) + \frac{1}{N} H(i),$$

avec I l'image, N le nombre total de pixels et $H(i)$ la valeur de l'histogramme pour l'intensité i.

On affecte ensuite $f(I) = c(I)$ pour un effet total ou $f(I) = \alpha c(I) + (1-\alpha)I$ pour un effet plus partiel.

↪ Implémentation OpenCV.JS

Contrast Locally Adaptive Histogram Equalization (CLAHE)



(a) (b) (c)

Figure 3.8 *Locally adaptive histogram equalization: (a) original image; (b) block histogram equalization; (c) full locally adaptive equalization.*

CLAHE

Le principe est ici de faire des égalisations d'histogrammes par bloc de taille fixe (*MxM*). Pour éviter, l'effet de blocs visibles, on calcule une fonction d'interpolation entre les différentes blocs. C'est la méthode CLAHE.

Filtrage linéaire

Filtrage linéaire

Les transformations précédentes ne prenaient en compte que l'intensité pure du pixel. Ici, l'idée est de tenir compte de l'environnement local du pixel (*i.e.* son voisinage).

- Filtres séparables (réduction de bruit avec filtre gaussien, netteté)
- Filtres orientables (détection de contours, de coins)

Il faudra définir pour éviter les effets de bords.

Filtrage linéaire

Convolution

L'idée principale est ici de définir un noyau (masque de dimensions impaires : 3*x*3, 5*x*5), *etc.*) et de le faire parcourir l'intégralité de l'image pixel par pixel.

45	60	98	127	132	133	137	133
46	65	98	123	126	128	131	133
47	65	96	115	119	123	135	137
47	63	91	107	113	122	138	134
50	59	80	97	110	123	133	134
49	53	68	83	97	113	128	133
50	50	58	70	84	102	116	126
50	50	52	58	69	86	101	120

$f(x,y)$

*

0.1	0.1	0.1
0.1	0.2	0.1
0.1	0.1	0.1

$h(x,y)$

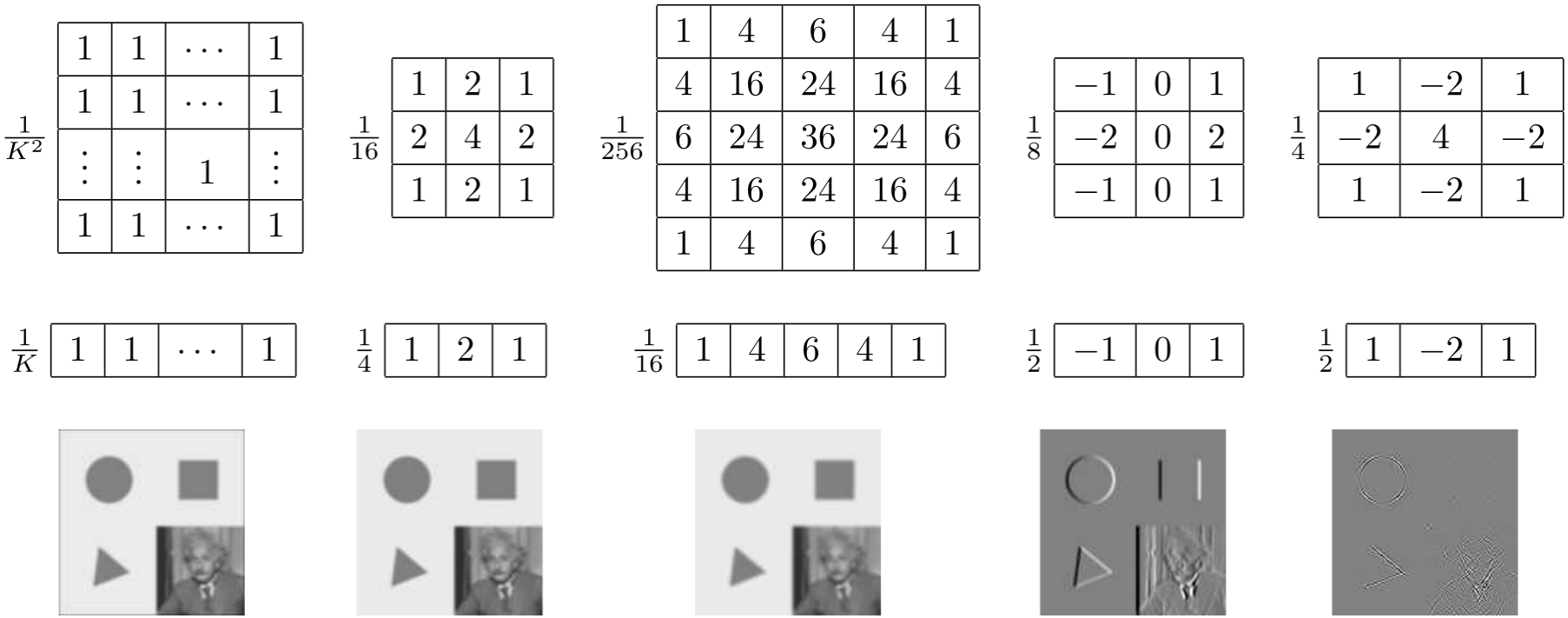
=

69	95	116	125	129	132
68	92	110	120	126	132
66	86	104	114	124	132
62	78	94	108	120	129
57	69	83	98	112	124
53	60	71	85	100	114

$g(x,y)$

Figure 3.10 *Neighborhood filtering (convolution): The image on the left is convolved with the filter in the middle to yield the image on the right. The light blue pixels indicate the source neighborhood for the light green destination pixel.*

Exemples de filtrage



(a) box, $K = 5$ (b) bilinear (c) "Gaussian" (d) Sobel (e) corner

Figure 3.14 *Separable linear filters: For each image (a)–(e), we show the 2D filter kernel (top), the corresponding horizontal 1D kernel (middle), and the filtered image (bottom). The filtered Sobel and corner images are signed, scaled up by 2× and 4×, respectively, and added to a gray offset before display.*

Filtrage non-linéaire

- Filtrage médian (Débruitage spécifique : poivre sel ou Poisson)
- Filtrage bilatéral et diffusion anisotropique (Débruitage avec préservation des contours)



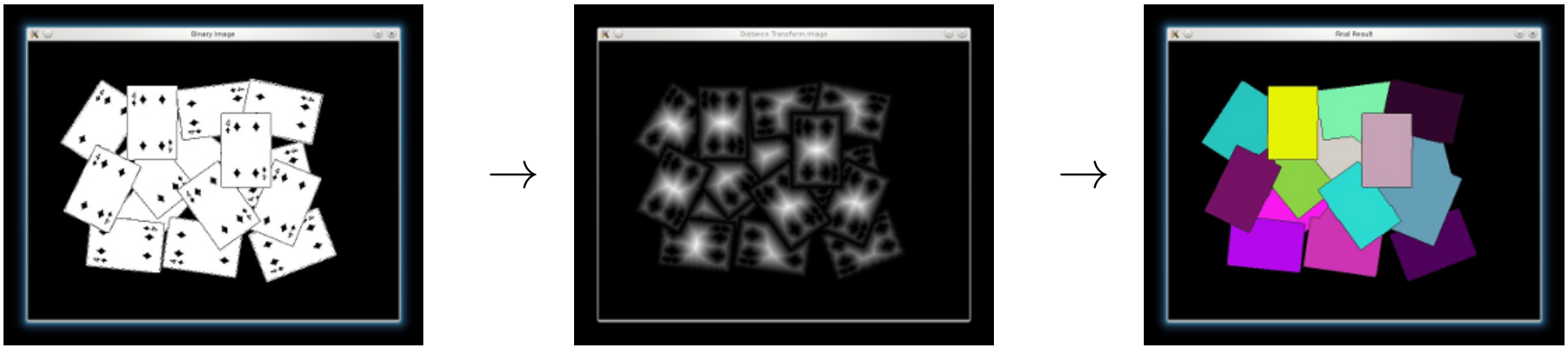



Traitements des images binaires

Traitements sur images binaires

Ces traitements s'opèrent en général après une première étape de segmentation (*e.g.* seuillage).

- Morphologie mathématique (change la forme des objets, affiner un masque de segmentation)
- Transformation en distance (mesure de distance entre des points de l'image)
- Componantes connexes (dénombrer les objets)



Morphologie mathématique

La morphologie mathématique s'opère uniquement sur des images binaires en ayant défini au préalable un élément structurant lui-aussi binaire. Il s'agit ensuite d'une simple convolution et le résultat va différer selon la nature de l'opération.

Opérations élémentaires

- Dilatation : Lorsque l'un des pixels blancs (1) de l'élément structurant intersecte un pixel blanc (1) de la forme, le centre de l'élément structurant sera blanc (1) dans l'image résultat.
- Erosion : Lorsque l'entièreté des pixels blancs (1) de l'élément structurant est contenu dans la forme de l'image, le centre de l'élément structurant sera blanc (1) dans l'image résultat.
- Ouverture : Dilatation $\circ$ Érosion
- Fermeture : Érosion $\circ$ Dilatation

Résultats morphologie mathématique



Figure 3.22 *Binary image morphology: (a) original image; (b) dilation; (c) erosion; (d) majority; (e) opening; (f) closing. The structuring element for all examples is a 5 × 5 square. The effects of majority are a subtle rounding of sharp corners. Opening fails to eliminate the dot, as it is not wide enough.*

Implémentation OpenCV.JS

Résultats morphologie mathématique



(a) (b) (c) (d) (e) (f)

Figure 3.22 *Binary image morphology: (a) original image; (b) dilation; (c) erosion; (d) majority; (e) opening; (f) closing. The structuring element for all examples is a 5 × 5 square. The effects of majority are a subtle rounding of sharp corners. Opening fails to eliminate the dot, as it is not wide enough.*

Implémentation OpenCV.JS

↪ Demo Fiji avec Cars.

1. Image → Adjust → Seuillage
2. Process → Binary → Érosion
3. Process → Image Calculator (AND)

Transformations de Fourier

Définition

La transformée de Fourier est une opération qui permet de passer d'une représentation spatiale d'une image (pixels) à une représentation fréquentielle de l'image (amplitude et phase pour les différentes fréquences présentes). On représente sur deux axes (fréquences horizontales f_x et fréquences verticales f_y) orthogonales les fréquences et leurs intensités.

Applications

- Analyse des textures (répétitions de motifs)
- Suppression du bruit basé sur sa fréquence (haute fréquence).
- Compression des données (JPEG utilise une transformée en cosinus)

Transformations de Fourier

Définition

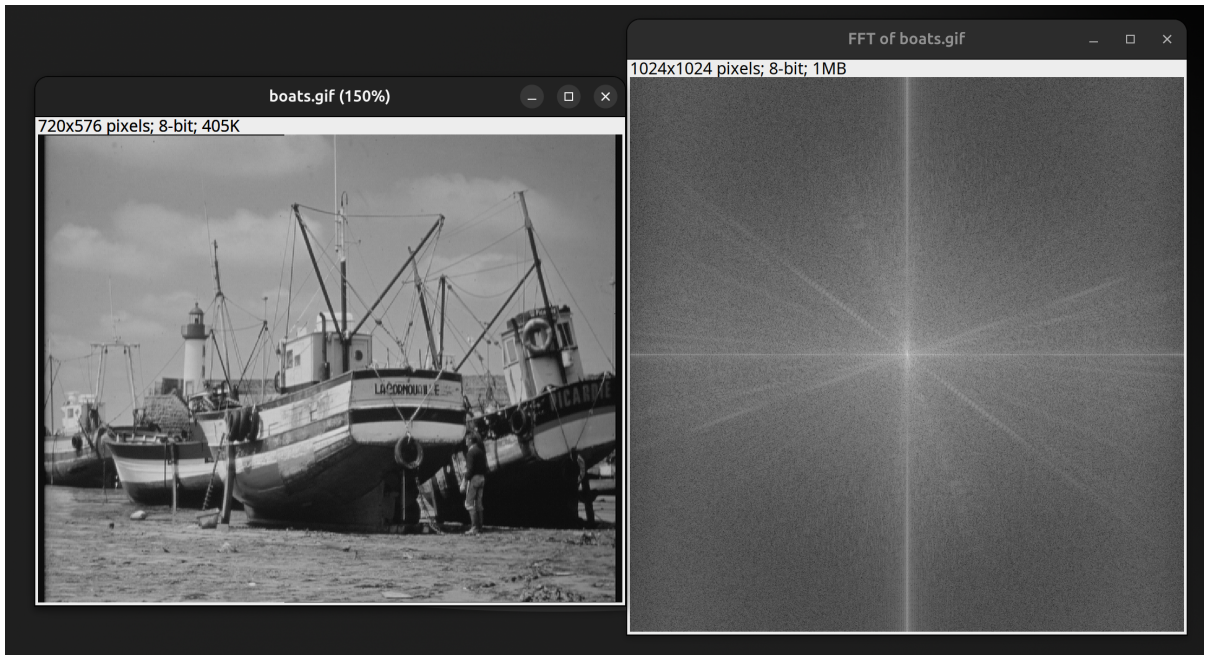
La transformée de Fourier est une opération qui permet de passer d'une représentation spatiale d'une image (pixels) à une représentation fréquentielle de l'image (amplitude et phase pour les différentes fréquences présentes). On représente sur deux axes (fréquences horizontales f_x et fréquences verticales f_y) orthogonales les fréquences et leurs intensités.



Transformations de Fourier

Définition

La transformée de Fourier est une opération qui permet de passer d'une représentation spatiale d'une image (pixels) à une représentation fréquentielle de l'image (amplitude et phase pour les différentes fréquences présentes). On représente sur deux axes (fréquences horizontales f_x et fréquences verticales f_y) orthogonales les fréquences et leurs intensités.



Transformations de Fourier

Définition

La transformée de Fourier est une opération qui permet de passer d'une représentation spatiale d'une image (pixels) à une représentation fréquentielle de l'image (amplitude et phase pour les différentes fréquences présentes). On représente sur deux axes (fréquences horizontales f_x et fréquences verticales f_y) orthogonales les fréquences et leurs intensités.

↪ Demo Fiji avec AuPbSn40.jpg

1. Image → FFT
2. Process → Smooth
3. Image → FFT

Pyramide et ondelettes

Pyramide multi-resolutions

L'idée ici est de représenter l'image à différentes résolutions spatiales pour pouvoir gagner en efficacité dans la recherche d'un objet dont on ne connaît pas forcément l'échelle spatiale (*e.g.* visage). Une fois, l'objet détecté à une échelle grossière, on peut l'affiner dans des niveaux d'échelles plus élevées.

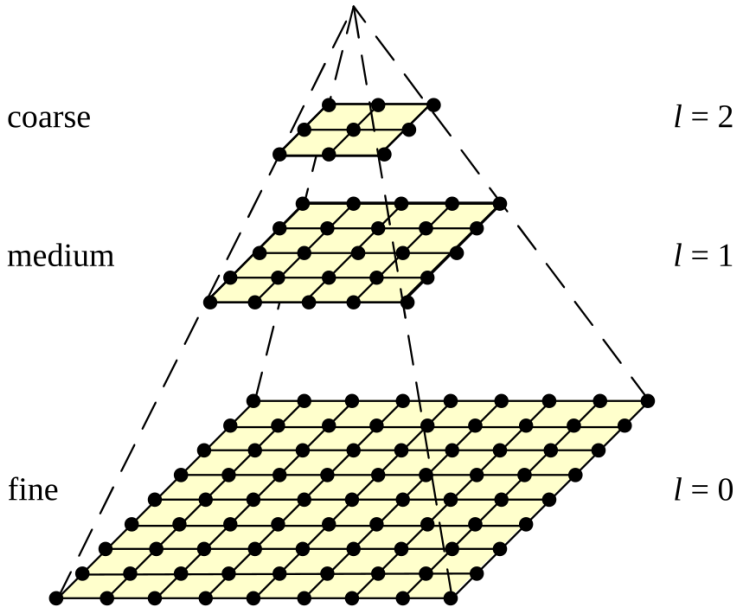
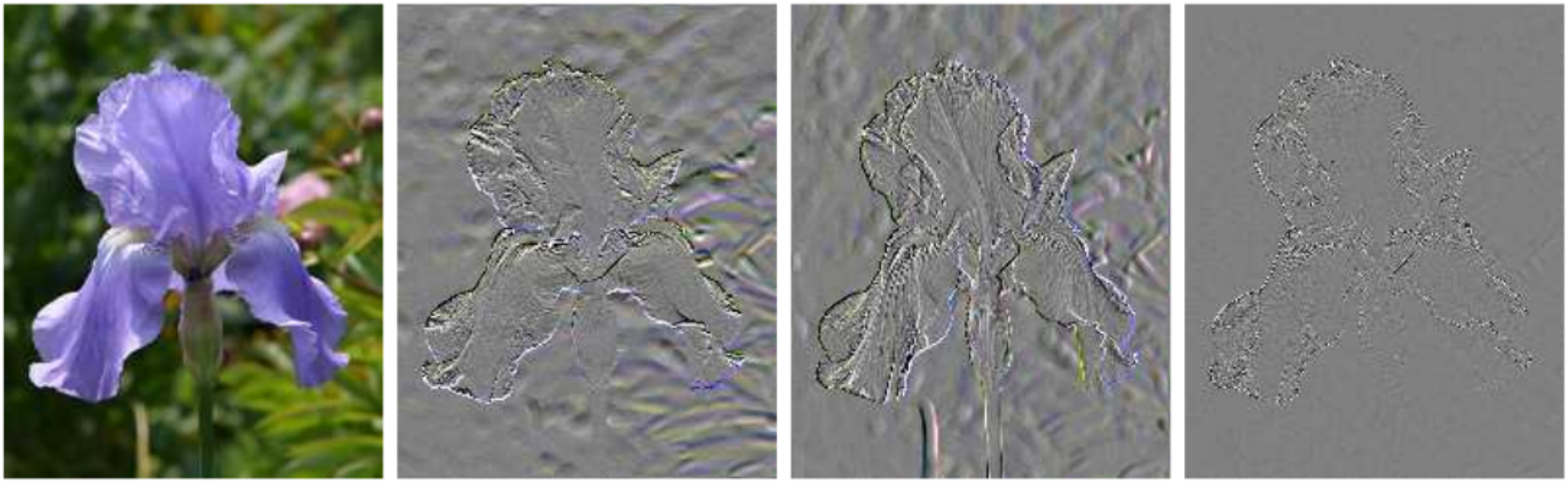


Figure 3.31 *A traditional image pyramid: each level has half the resolution (width and*

Pyramide et ondelettes

Ondelettes

L'idée est similaire sauf qu'ici on décompose l'image suivant des ondelettes (filtres) qui localise le signal dans l'espace et en fréquences.



Transformations géométriques

Les transformations géométriques ne vont pas altérer les intensités dans l'image, ni sa résolution mais on va modifier son domaine ("grille").

Transformations paramétriques

Ces transformations sont contrôlées par un ensemble de paramètres.

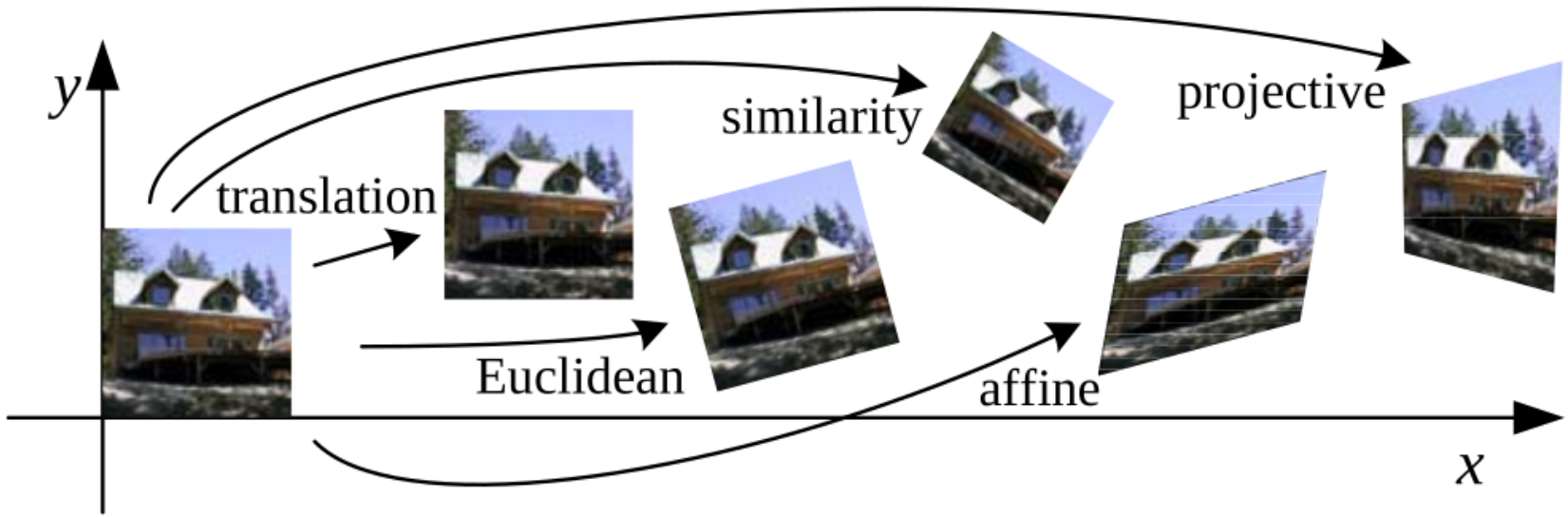


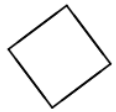
Figure 3.44 *Basic set of 2D geometric image transformations.*

Transformations géométriques

Les transformations géométriques ne vont pas altérer les intensités dans l'image, ni sa résolution mais on va modifier son domaine ("grille").

Transformations paramétriques

Ces transformations sont contrôlées par un ensemble de paramètres.

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2\times3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2\times3}$	3	lengths	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2\times3}$	4	angles	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2\times3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3\times3}$	8	straight lines	

Transformations géométriques

Les transformations géométriques ne vont pas altérer les intensités dans l'image, ni sa résolution mais on va modifier son domaine ("grille").

Transformations paramétriques

Ces transformations sont contrôlées par un ensemble de paramètres.

↪Implémentation OpenCV.JS

Bibliothèque



OpenCV

OpenCV is open source, contains over 2500 algorithms, and is operated by the non-profit Open Source Vision Foundation. Since June 2000.

Plateformes

It has C++, Python, Java and MATLAB interfaces and supports Windows, Linux, Android and Mac OS. La bibliothèque est également présente sous la forme javascript.

Organisation de la librairie

- Main modules:
  - core. **Core functionality**
  - imgproc. **Image Processing**
  - imgcodecs. **Image file reading and writing**
  - videoio. **Video I/O**
  - highgui. **High-level GUI**
  - video. **Video Analysis**
  - calib3d. **Camera Calibration and 3D Reconstruction**
  - features2d. **2D Features Framework**
  - objdetect. **Object Detection**
  - dnn. **Deep Neural Network module**
  - ml. **Machine Learning**
  - flann. **Clustering and Search in Multi-Dimensional Spaces**
  - photo. **Computational Photography**

Exemple simple : chargement d'image

```
1  import cv2 as cv
2  import sys
3
4  img = cv.imread("images/Satellite.jpg")
5
6  if img is None:
7          sys.exit("Could not read the image.")
8
9  cv.imshow("Display window", img)
10 k = cv.waitKey(0)
11
12 if k == ord("s"):
13         cv.imwrite("starry_night.png", img)
```

Exemple simple : flux video

```python
import cv2 as cv

cap = cv.VideoCapture(0)
if not cap.isOpened():
    print("Cannot open camera")
    exit()

while cv.waitKey(1) != ord('q'):
    # Capture frame-by-frame
    ret, frame = cap.read()

    # if frame is read correctly ret is True
    if not ret:
        print("Can't receive frame (stream end?). Exiting ...")
        break
    else:
        # Our operations on the frame come here
        gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
        # Display the resulting frame
        cv.imshow('frame', gray)
```

Exemple simple : Valeur des pixels

```python
# pixel value
import cv2 as cv
import numpy as np

img = cv.imread("images/Satellite.jpg")

x, y = 10, 10
px = img[x,y]
print(f"Valeur du pixel pour les coordonnées en BGR ({x},{y}): {px}")
```

Exemple simple : Espace de couleurs

```
7  # Créer une image noire de 256x256 pixels
8  largeur, hauteur = 128, 128
9  image = np.zeros((hauteur, largeur, 3), dtype=np.uint8)
0
1  image[:,:,0] = np.full((hauteur, largeur), 100)
2  image[:,:,1] = np.full((hauteur, largeur), 0)
3  image[:,:,2] = np.full((hauteur, largeur), 0)
4
5  # Couleur uniforme ? Laquelle ?
6  cv.imshow('Couleur',image)
7  cv.waitKey(0)
8  cv.destroyWindow("Couleur")
```

Exemple simple : Espace de couleurs

```python
# pixel value
import cv2 as cv
import numpy as np

img = cv.imread("images/Satellite.jpg")

x, y = 10, 10
px = img[x,y]
print(f"Valeur du pixel pour les coordonnées en BGR ({x},{y}): {px}")

# Changement de l'espace de couleurs
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
px = img_rgb[x,y]
print(f"Valeur du pixel pour les coordonnées en RGB ({x},{y}): {px}")
```

Exemple simple : conversion d'espaces

```python
import cv2 as cv

img = cv.imread("images/Satellite.jpg")

#Transformation de l'espace couleurs
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
img_rgba = cv.cvtColor(img, cv.COLOR_BGR2RGBA)
# img_hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
# img_bgr2 = cv.cvtColor(img_hsv, cv.COLOR_HSV2BGR)
img_gray1 = cv.cvtColor(img,cv.COLOR_BGR2GRAY)
img_gray2 = cv.cvtColor(img,cv.COLOR_RGB2GRAY)

for line in img_rgba[:,:]:
    for px in line:
        px[3]=128;

print(img_rgba.shape," ",img_rgba[100,100])

# cv.imshow("Fenetre BGR", img)
# cv.imshow("Fenetre RGB", img_rgb)
```

Sauvegarde d'images

```python
import cv2 as cv

img = cv.imread("images/Satellite.jpg")

# Enregistre l'image en se basant sur l'extension de fichier
cv.imwrite("images/satellite.png",img)

# Ajoute un canal de transparence
img_bgra = cv.cvtColor(img,cv.COLOR_BGR2BGRA)

# réduit la transparence de moitié
img_bgra[:,:,3] = 128
# for line in img_bgra[:,:]:
#     for px in line:
#         px[3]=128;

# Enregistre l'image en se basant sur l'extension de fichier
cv.imwrite("images/satellite_a.png",img_bgra)

#In general, only 8-bit unsigned (CV_8U) single-channel or 3-channel
```

Exemple simple : décomposition des canaux de couleurs

```
1  import cv2 as cv
2  from matplotlib import pyplot as plt
3
4  # cv.IMREAD_UNCHANGED permet de conserver le canal de transparence
5  imga = cv.imread("images/opencv-logo.png", cv.IMREAD_UNCHANGED)
6
7  #Extraction de chacun des canaux de couleurs
8  bc, gc, rc, ac = cv.split(imga)
9
10 # plt.imshow() attend des images dans l'ordre RGB.
11 img = cv.cvtColor(imga, cv.COLOR_BGRA2RGBA)
12 plt.subplot(151), plt.imshow(img), plt.title("original")
13
14 # Version devinette
15 plt.subplot(152), plt.imshow(bc, 'gray')
16 plt.subplot(153), plt.imshow(gc, 'gray')
17 plt.subplot(154), plt.imshow(rc, 'gray')
18 plt.subplot(155), plt.imshow(ac, 'gray')
19
20 # Version corrigée
```

Références

- Computer Vision : Algorithms and Applications. Szeliski R., 2nd edition, Springer, 2022.
- Digital Image Processing. Gonzales R. and Woods R., 4th Edition, Pearson, 2018.
- Documentation OpenCV, 2025.